ECE 385

SPRING 2025

Experiment 4

Introduction to SystemVerilog, FPGA, CAD, and 16-bit Adders

Artha Amanda Sidauruk
(3220300399)
Cristina Natalia Lumban Tobing
(3220300398)
Friday, Mar 12, 2025. D-231
TA: Qiyang Wu

1 Introduction

This lab is divided into two big tasks. The first task is to extend a 4-bit logic processor to an 8-bit logic processor. The 8-bit logic processor contains a control unit, register unit, computation unit, routing unit, and hex driver unit. In principle, the register unit contains two 8-bit registers. The control unit provides the control signal to the register unit. The computation unit executes the logical computation (i.e., bitwise operations), which is determined by 3-bit signal F. After the computation, the routing unit is responsible for routing the signals back to the register unit. The corresponding direction routes are determined by a 2-bit signal R. Finally, the hex driver unit displays the data. The main changes from a 4-bit serial logic processor to an 8-bit logic processor are the register structure, control unit (i.e., the finite state machine), and the computation unit.

The second task is to build the 16-bit carry-ripple adder, a 16-bit carry-lookahead adder, and a 16-bit carry-select adder in System Verilog. All the adders aim to add the two inputs of 16-bit but with different implementations. The CRA (Carry-Ripple Adder) is the simplest and the most straightforward adder compared to CLA (Carry-Lookahead Adder) and CSA (Carry-Select Adder), but it takes a longer time for computation. CRA is built by 16 full adders. CLA minimizes the computation time by utilizing the generating (G) and propagating (P) logic concepts. Specifically, it predicts the carry-out for any corresponding input and is built in a hierarchical design. CSA works by doing the computation for both possible values of carry-in (i.e., '0' and '1') and uses the multiplexer to choose the final sum for the actual carry-in value.

2 Part 1 – Serial Logic Processor

2.1 Top Level Schematic Generated by the RTL Viewer

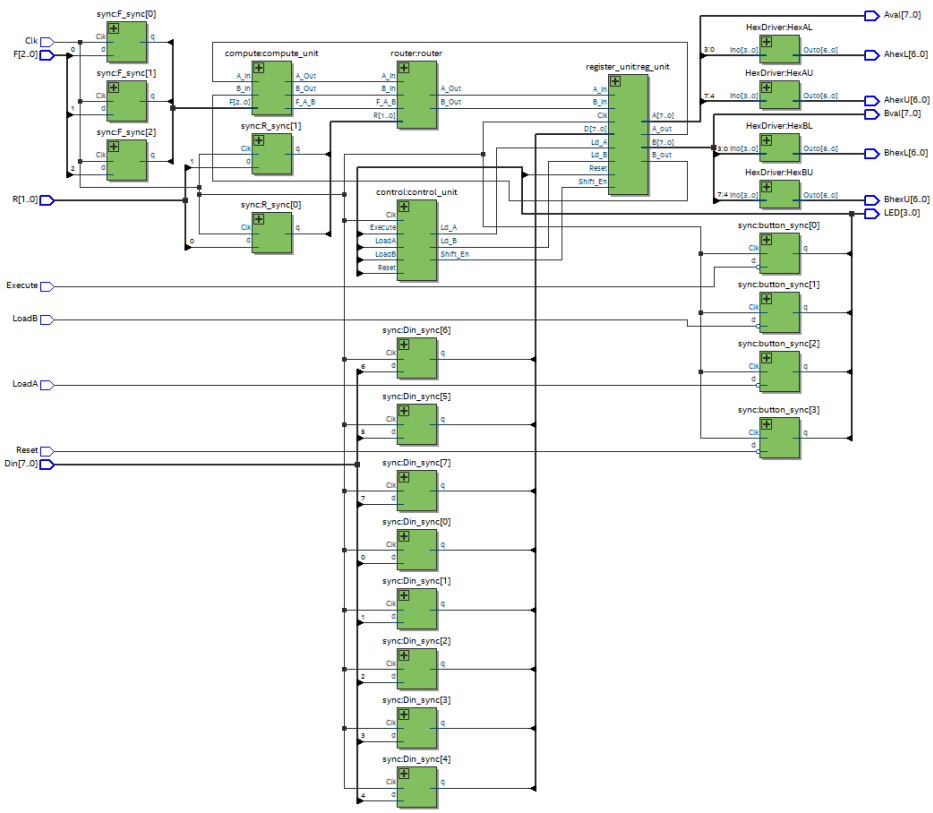

Figure 1: RTL Viewer of 8-bit Logic Processor

2.2 Description of Extending a 4-bit Logic Processor to an 8-bit Version

The following are changes to the code made to extend a 4-bit logic processor to an 8-bit version:

- In the given 4-bit logic processor code, registers can store 4-bit data. However, to build an 8-bit logic processor, the register must be able to store 8-bits. So, changes were made to the code by changing the 4-bit wide input/output to 8-bit wide input/output.
- The control unit also changed, specifically the number of states. A 4-bit logic processor has 6 utilized states. We need to extend it by adding 4 additional states and implementing what the next state should do.
- Extend the hexdriver so it can successfully view the upper nibble of the registers.
- Changed the testbench to generate the simulated waveform of an 8-bit logic processor correctly.

2.3 Simulation of the 8-bit Logic Processor

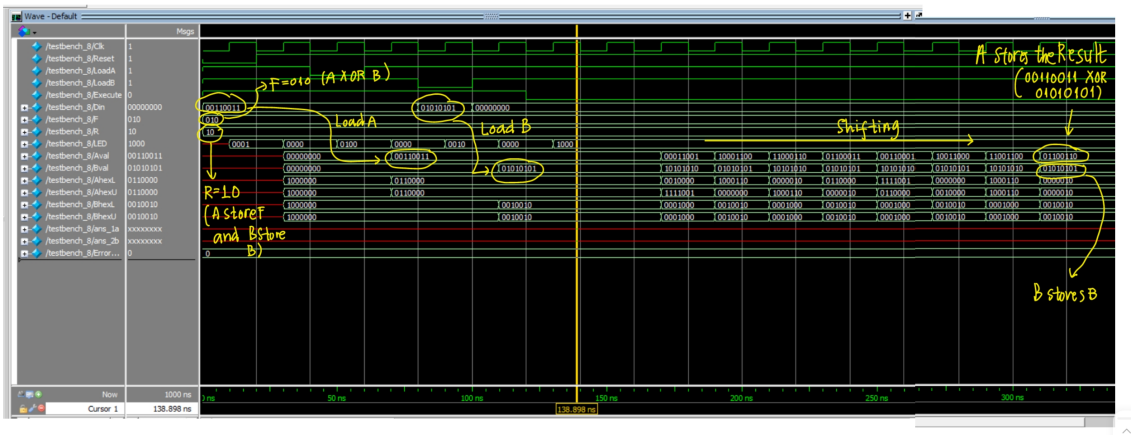

Figure 2: First Operation

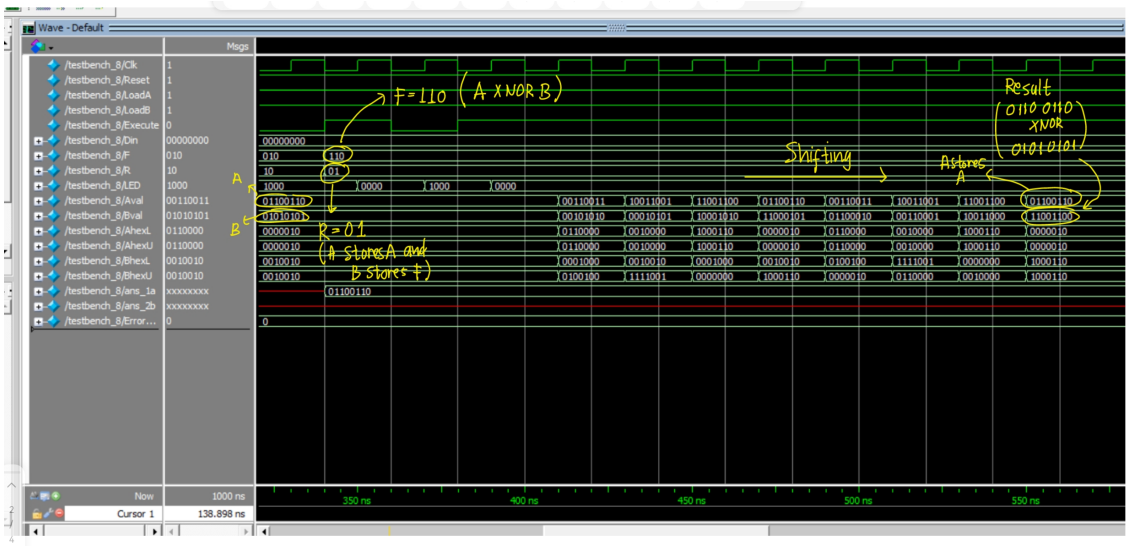

Figure 3: Second Operation

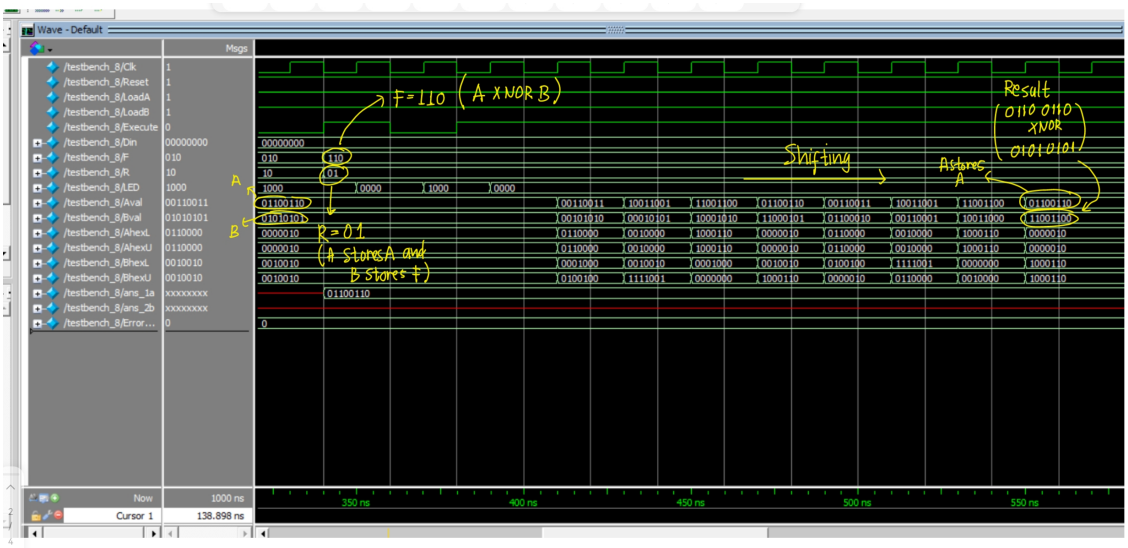

Figure 4: Third Operation

3 Part 2 – Adders

Here is the RTL Viewer of 16-bit adders. The "ripple_adder:ripple_adder_inst" section also shows the position of the Carry-Lookahead Adder and Carry-Select Adder instances according to the context lab4_adders_toplevel.sv file, where only one adder can be simulated at a time.

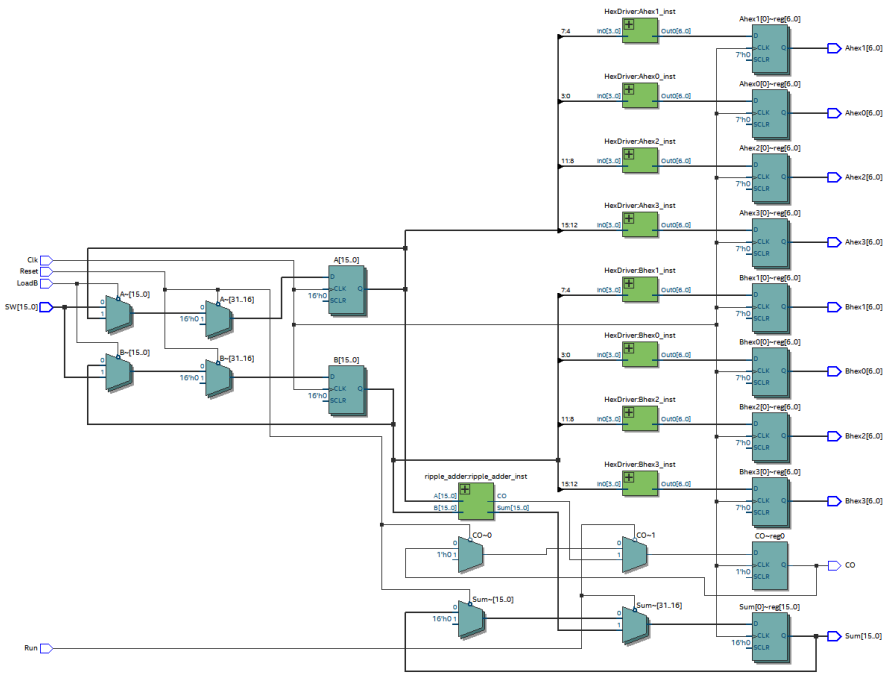

Figure 5: 16-bit Adders

3.1 Ripple Carry Adder

3.1.1 Written Description of the Architecture of CRA

The 16-bit ripple carry adder comprises 16 full adders connected in cascaded form. Carry-out of each full adder is fed to the next stage full adder as a carry-in. In this lab, a 16-bit ripple carry adder comprises four 4-bit full adder modules connected in series through the carry bits. The inputs are 16 bits of data, which are $A[15:0]$ and $B[15:0]$, divided into four 4-bit segments. Each segment is fed into a 4-bit full adder (FA4B0, FA4B1, FA4B2, and FA4B3). FA4B0 is the least significant 4-bit full adder; it has *carry_in* (*cin*) = 0 and will output the *carry_out* (*cout*) that will propagate continuously to become the *carry_in* for the next stage 4-bit full adder until finally, the last 4-bit full adder computes the most significant 4-bit sum and the last *carry_out* (*CO*).

No less important, the sum of the entire computation is the 16-bit $Sum[15:0]$ that is concatenated at the end of the process. The drawback of the ripple carry adder is that its propagation delay increases with the number of bits due to the "waiting time" for the lowest full adder stage to produce *carry_out*. Hence, this type of adder is slower than other adders, which will be explained in the next section.

3.1.2 Block Diagram

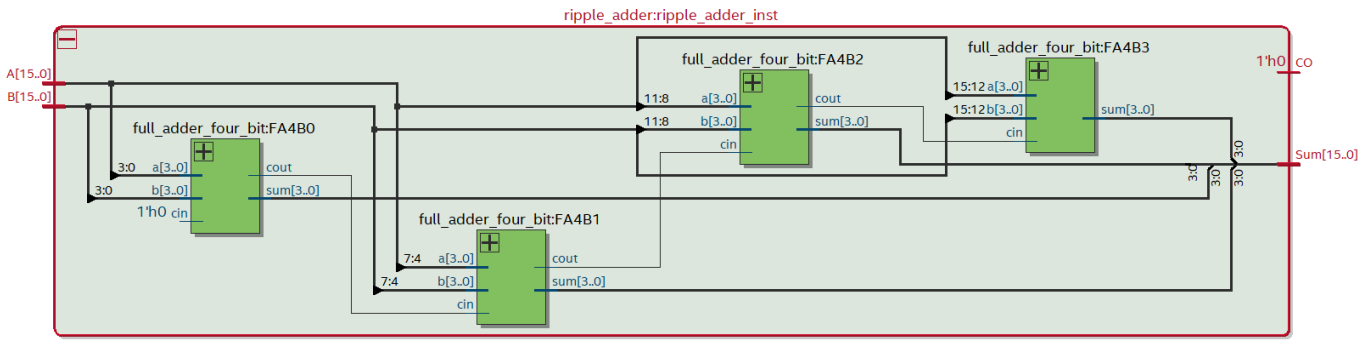

Figure 6: 4x4-bit Carry-Ripple Adder

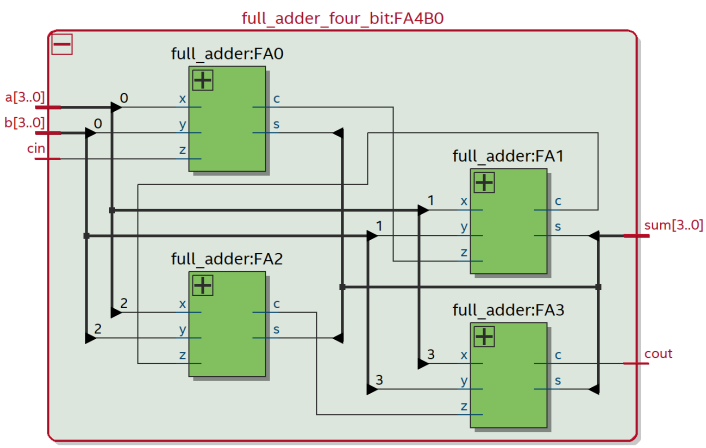

Figure 7: 4-bit Full-Adder

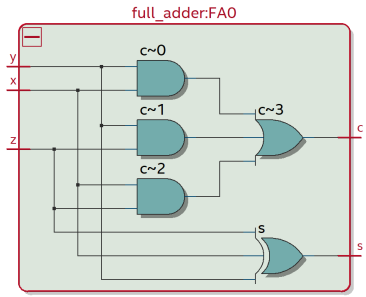

Figure 8: 1-bit Full-Adder

3.2 Carry Lookahead Adder (CLA)

3.2.1 Written Description of the Architecture of CLA

- **Description of P and G Logic**

  To minimize the delay time, the carry-lookahead adder applies the concept of generating (G) and propagating (P) logic. Since the delay time in the ripple carry adder is mostly due to the "waiting time" for the previous full-adder stage to produce carry-out, the carry-lookahead adder solves this problem through "prediction of carry-out" using P and G logic.

  Each full adder that builds the 16-bit carry-lookahead adder produces P and G. The P and G logics, along with the initial carry-in (C_{in}, usually 0), are used to determine the carry-in for the next stage of the full adder. P and G fully depend on the current inputs, A and B. The boolean expression of P is $P = A \cdot B$, and the boolean expression of G is $G = A \oplus B$. Then, the carry-out is determined by $C_{i+1} = G_i + (P_i \cdot C_i)$, where C_i is expanded in terms of P_i and G_i as shown below:

$$
\begin{aligned}
C_0 &= C_{in} \\
C_1 &= C_{in} \cdot P_0 + G_0 \\
C_2 &= C_{in} \cdot P_0 \cdot P_1 + G_0 \cdot P_1 + G_1 \\
C_3 &= C_{in} \cdot P_0 \cdot P_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + G_1 \cdot P_2 + G_2 \\
&\vdots
\end{aligned}
$$

  So, $C1$ will be a carry-in for the second adder, $C2$ will be a carry-in for the third adder, and so on. This way, the computation time will increase compared to the ripple carry adder.

- **Partition of the Adder** To achieve the 16-bit carry-lookahead adder, a 4x4-bit carry-lookahead adder is designed instead. By constructing a 4x4 CLA, we simplify the design, reduce the propagation delay, and make the adder for the larger bit widths practically possible.

  The 16-bit inputs A[15:0] and B[15:0] are divided into four 4-bit segments, each of which becomes an input for 4-bit CLA.

  The 4-bit CLA consists of four full adders and a 4-bit Carry-Lookahead Unit. Carry-Lookahead Unit is responsible for computing "carry-out prediction": $C1$, $C2$, $C3$; group propagate: P_G; the group generate: G_G. The boolean expressions are the following:

$$
\begin{aligned}
P_G &= P_0 \cdot P_1 \cdot P_2 \cdot P_3 \\
G_G &= G_3 + G_2 \cdot P_3 + G_1 \cdot P_3 \cdot P_2 + G_0 \cdot P_3 \cdot P_2 \cdot P_1
\end{aligned}
$$

Four 4-bit CLAs that are connected in series and a 16-bit Lookahead Carry Unit together construct a 4x4-bit CLA. The carry-in for the first 4-bit CLA is the initial Cin (set to 0), and the carry-in for the next three 4-bit CLAs is predicted using group propagate (P_G) and group generate (G_G) of each previous 4-bit CLA in question. Below is the Boolean expression for carry-in of the other three 4-bit CLAs:

$$
\begin{aligned}
C_4 &= G_{G0} + C_0 \cdot P_{G0} \\
C_8 &= G_{G4} + G_{G0} \cdot P_{G4} + C_0 \cdot P_{G0} \cdot P_{G4} \\
C_{12} &= G_{G8} + G_{G4} \cdot P_{G8} + G_{G0} \cdot P_{G8} \cdot P_{G4} + C_0 \cdot P_{G8} \cdot P_{G4} \cdot P_{G0}
\end{aligned}
$$

Where $C4$ is the carry-in for the second 4-bit CLA, $C8$ is the carry-in for the third 4-bit CLA, and $C12$ is the carry-in for the last 4-bit CLA. In addition, the 16-bit Lookahead Carry Unit also produces the carry-out(CO) of the entire adder by using a boolean expression: $CO = G_{G12} + G_{G8} \cdot P_{G12} + G_{G4} \cdot P_{G12} \cdot P_{G8} + G_{G0} \cdot P_{G12} \cdot P_{G8} \cdot P_{G4} + C_0 \cdot P_{G12} \cdot P_{G8} \cdot P_{G4} \cdot P_{G0}$.

3.2.2 Block Diagram

- **4-bit CLA**

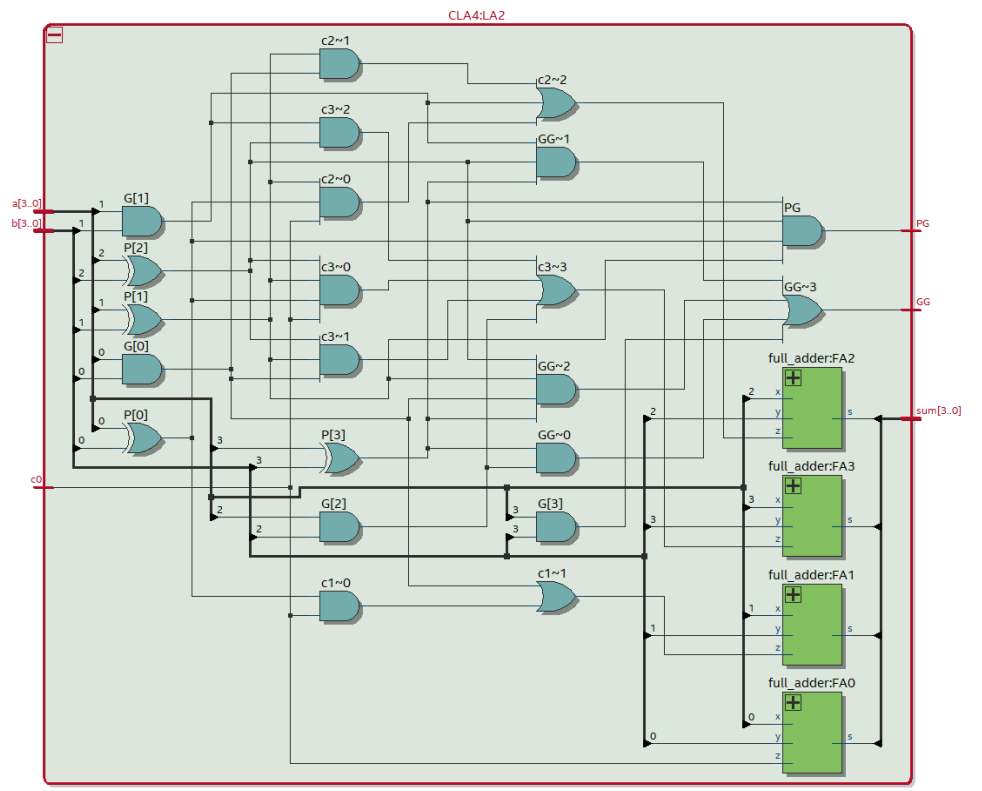

Figure 9: 4-bit Carry-Lookahead Adder

- **16-bit CLA**

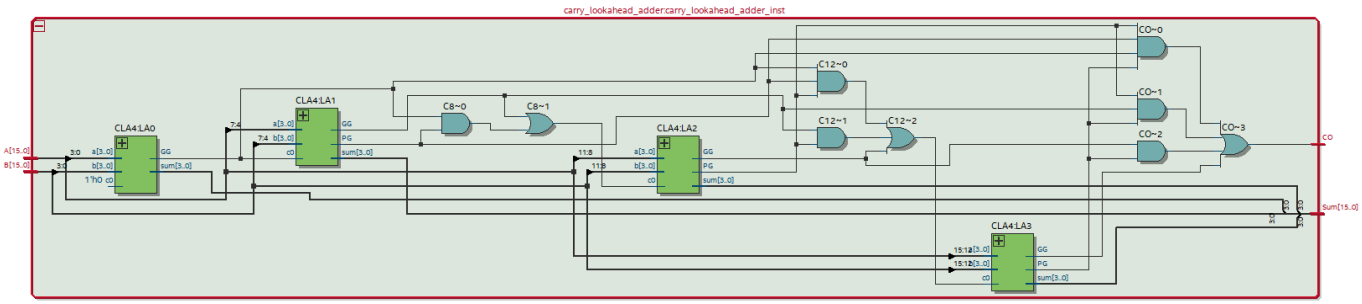

Figure 10: 4x4-bit Carry-Lookahead Adder

3.3 Carry Select Adder (CSA)

3.3.1 Written Description of the Architecture of the CSA

The main principle of how the carry select adder (CSA) works is speculatively computing multiple sums in parallel by pre-computing the value of sum and carry-out for two scenarios (i.e., all the possible values of carry-in). The first scenario is setting the carry-in to be 1, and the second scenario is setting the carry-in to be 0. In the end, when the actual carry-in is known, the final value of the sum and carry-out is determined by using a 2x1 multiplexer. The 2x1 MUX rapidly chooses one of the two calculated sums and carry-outs according to the actual carry-in. This mechanism reduces the propagation delay and achieves a high speed of computation.

The implementation of a 4x4-bit hierarchical structure designs a 16-bit CSA. The 16-bit inputs A[15:0] and B[15:0] are divided into 4 segments of 4-bit. Each segment is fed into a 4-bit CSA as an input. The first 4-bit CSA only consists of one 4-bit full adder since the initial carry-in is always assumed to be 0. The other three 4-bit CSAs consist of two 4-bit full adders, where one 4-bit full adder computes the sum and carry-out for carry-in = 0 and the other for carry in=1. In each of the three 4-bit CSAs, there are five 2x1 multiplexer modules. Four MUXs work to choose a 4-bit sum[3:0], and one MUX works to choose the carry-out. The selection input for all the 2x1 MUX is the actual carry-in.

These four CSAs are connected in series since the carry-out of the previous 4-bit CSA becomes the carry-in for the next 4-bit CSA. Overall, a 16-bit Sum[15:0] is obtained from the concatenation of a 4-bit sum from each 4-bit CSA. Then, carry-out (CO) is the carry-out of the last CSA.

3.3.2 Block Diagram

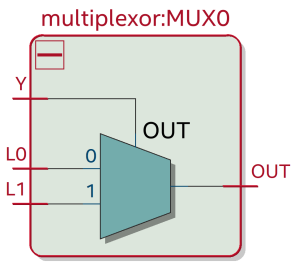

Figure 11: 2x1 Multiplexer

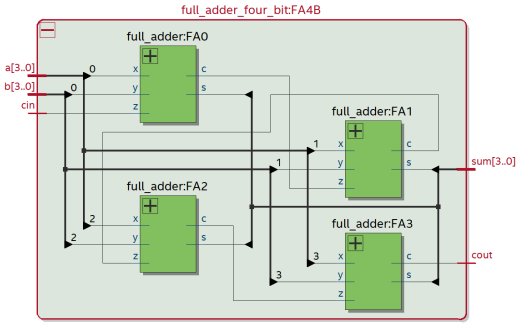

Figure 12: First 4-bit Carry-Select Adder

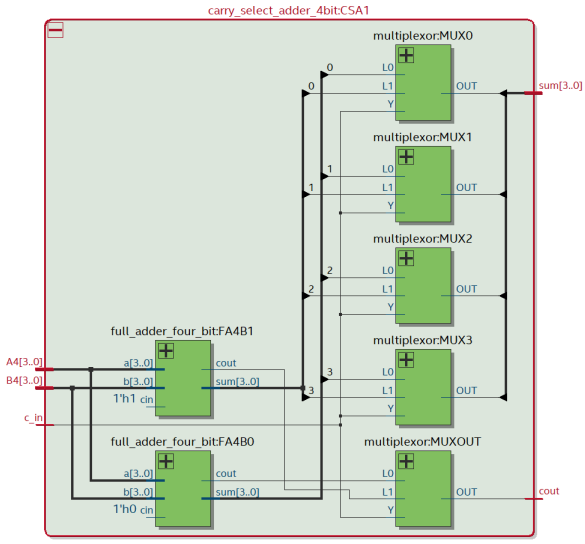

Figure 13: The Other Three 4-bit Carry-Select Adder

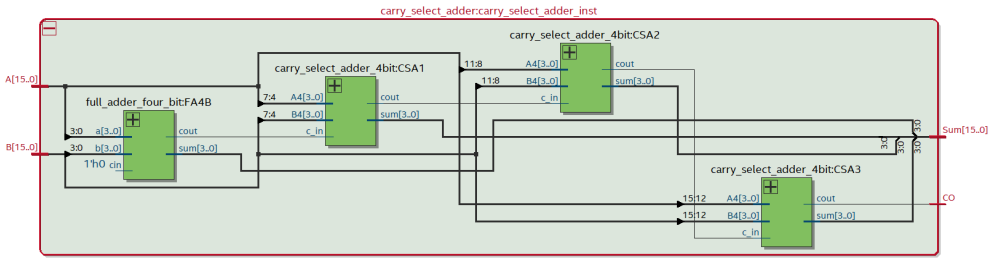

Figure 14: 4x4 Carry-Select Adder

3.4 Written Description of All .sv Modules

Module: lab4_adders_toplevel
Inputs: Clk, Reset, LoadB, Run, [15 : 0] SW
Outputs: CO, [15 : 0] Sum, [6 : 0] Ahex0, Ahex1, Ahex2, Ahex3, Bhex0, Bhex1, Bhex2, Bhex3
Description: This is the top-level adder. The module connects the adder design with the external inputs (e.g., buttons and switches) and outputs (e.g., hex displays and LEDs) in the FPGA board.
Purpose: Manage all inputs, outputs, and control inputs. Implementing addition displays the result of the design in the FPGA board.

Module: ripple_adder
Inputs: [15 : 0] A, [15 : 0] B
Outputs: Sum, CO
Description: This is a 16-bit ripple carry adder built by 4x4-bit full adder (i.e., utilizes full_adder_four_bit module). Four 4-bit full adders are connected in series through passing carry-out sequentially.
Purpose: To add 16-bit inputs, A and B, along with carry-in bit, and compute the corresponding sum and carry-out bit.

Module: full_adder
Inputs: x, y, z
Outputs: s, c
Description: This is the 1-bit full adder. This module uses XOR gate to compute the sum and uses AND & OR gates to compute the carry-out bit.
Purpose: To add 1-bit inputs, x and y, along with carry-in bit, and compute the corresponding sum and carry-out bit.

Module: full_adder_four_bit
Inputs: [3 : 0] a, [3 : 0] b, cin
Outputs: [3 : 0] sum, cout
Description: This is the 4-bit full adder built from four 1-bit full adders (i.e., utilizes full_adder module). Four 1-bit full adders are connected in series through passing carry-out sequentially.
Purpose: To add 4-bit inputs, a and b, along with carry-in bit, and compute the corresponding sum and carry-out bit.

Module: carry_lookahead_adder
Inputs: [15 : 0] A, [15 : 0] B
Outputs: [15 : 0] Sum, CO
Description: This is a 16-bit carry lookahead adder built using four 4-bit carry lookahead adder (i.e., utilizes CLA4 module). The adder uses group propagate (PG) and group generate (PG) logic to compute intermediate carries (C_4, C_8, C_12) and the final carry-out (CO). There values are computed using the Boolean expression that is explained in the previous section of this report.
Purpose: To add 16-bit inputs, A and B, using the carry lookahead concept. The operation using PG and GG logic to reducing the propagation delay by prediction of carry bits.

Module: CLA4
Inputs: [3 : 0] a, [3 : 0] b, c0

Outputs: [3 : 0] sum, PG, GG
Description: This is a 4-bit carry lookahead adder built from four 4-bit full adder (i.e., utilizes full_adder module). The adder use propagated (P) and generated (G) logic concept to compute the carry-in for three 4-bit full adders. In this module, the "generated" logic: G; the "propagated" logic: P; the "carry-out prediction": c1, c2, c3; the "group propagate" logic: PG; and the "group generate" logic: GG, are computed using the Boolean expression that is explained in the previous section of this report.
Purpose: To perform the addition of two 4-bit inputs efficiently. Uses the operation using P and G logic to maximize the operation's speed.

Module: carry_select_adder
Inputs: [15 : 0] A, [15 : 0] B
Outputs: [15 : 0] Sum, CO
Description: This is a 4x4-bit carry-select adder. To achieve the hierarchical model of CSA, this module utilizes one full_adder_four_bit module for the lowest significant group input and three carry_select_adder_4bit modules for the rest.
Purpose: To obtain the addition result from dual 16-bit inputs. Time efficiency is improved by performing the addition twice: once for carry-in = 0 and carry-in = 1. A multiplexer (mux) selects the final result according to the actual carry-in value.

Module: multiplexor
Inputs: L0, L1, Y
Outputs: OUT
Description: This is the 2x1 multiplexer that accepts three 1-bit inputs and produces a 1-bit output. L0 and L1 are the inputs to be selected, Y is a control (select) signal, and OUT is the output signal. The Boolean expression of OUT is $OUT = L_0S' + L_1S$.
Purpose: This module is intended to select the pre-computed cout and sum that correspond to the actual cin in a 16-bit carry select adder operation.

Module: carry_select_adder_4bit
Inputs: [3 : 0] A4, [3 : 0] B4, c_in
Outputs: [3 : 0] sum, cout
Description: This is the 4-bit carry-select adder. Two full_adder_four_bit modules are used to perform the addition, with one adder assuming carry-in = 0 and the other assuming carry-in = 1. Then, a multiplexor module is used to select the sum and carry-out based on the actual carry-in.
Purpose: This module aims to obtain the sum of two 4-bit inputs. Pre-computed addition reduces time delay propagation.

Module: HexDriver
Inputs: [3 : 0] In0
Outputs: [6 : 0] Out0
Description: This module controls the Hex LEDs on the FPGA board. By using this module, the values stored in the register appear on the board, and addition can be performed.
Purpose: To display the input and output in the FPGA board. Allows us to check and confirm the function of the adder.

Module: testbench
Inputs: -
Outputs: -

Description: This module applies test vectors and compares the output with the correct result.
Purpose: To verify the correctness of the function of the adder and detect any unintended output.

3.5 A High-Level Overview of the Area, Complexity, and Performance Tradeoffs Between the Adders

- **Carry-Ripple Adder (CRA)**
  - **AREA** CRA is the most straightforward adder to implement compared to CLA and CSA. CRA consists of full-adders, where each full-adder only requires gates for sum and carry-out calculations. Thus, the area required is relatively small. However, if the bid-width increases, the area size will grow linearly.
  - **COMPLEXITY** 16-bit CRA has a simple design since it is built by chaining four 4-bit full-adders together. Thus, CRA has the least complexity.
  - **PERFORMANCE**
    CRA tends to be slow with time complexity O(N). The carry bit has to ripple through all the stages of the adders. Hence, the propagation delay of CRA increases with the bid-width input data.
- **Carry-Lookahead Adder (CLA)**
  - **AREA** The area of CLA is larger than the CRA. The additional area is needed to calculate the P and G logic. The growth of the area is directly proportional to the bit-width.
  - **COMPLEXITY** CLA is the most complex adder compared to CRA and CSA. To "predict" the carry bits, more logic expressions and units are used, resulting in a more complex implementation and design.
  - **PERFORMANCE** CLA is the fastest adder compared to CRA and CSA. The implementation of propagated (P) and generated (G) logics is an excellent solution to reduce propagation delay time. By predicting carry bits, much time is saved. In the ideal conditions (e.g., no delay of the gates), the time complexity of CLA is $O(1)$. However, in real-world design, the actual implementation has $O(\log(N))$ complexity.
- **Carry-Select Adder (CSA)**
  - **AREA** CSA works by pre-computing sum and carry-out for two possible values of carry-in (i.e., carry-in = 0 and carry-in = 1) and using the 2x1 MUX to select the output based on the actual value of carry-in. So, the area used is approximately doubled. The area grows as the bid width increases.
  - **COMPLEXITY** Complexity analysis for N-bit carry-select adders gives $O(\sqrt{N})$. The complexity occurs due to the involvement of multiplexers and pre-computed mechanisms.
  - **PERFORMANCE** The CSA can speed up the carry computation since it can compute the carry in parallel for different assumptions. However, for large N, overhead may occur. The CSA's speed is smaller than the CLA's and larger than the CRA's. It still needs to wait for the carry bits from the intermediate stage.

- **Area-Complexity-Performance Tradeoffs**
  - **CRA** Best for use on small bit-widths because of its simple design and the area required is not too large. However, it is not good for larger bit-widths due to the carry propagation delay which causes its performance to decrease.
  - **CLA** Has very good performance compared to CRA and CSA, so it is good for use on larger bit-widths. However, carry generation and propagation increase complexity and the amount of area required.
  - **CSA** Has a balance of area and performance, so it is good for use on medium to large bit-widths due to faster carry computation. However, multiplexer and adder units increase complexity and the amount of area required.

3.6 Performance of Each Adder

	Carry-Ripple	Carry-Select	Carry-Lookahead
Memory (BRAM)	0	0	0
Frequency	59.45 MHz	78.9 MHz	79.42 MHz
Total Power	156.86 mW	162.62 mW	162.26 mW

Figure 15: Table of Performance of Each Adder

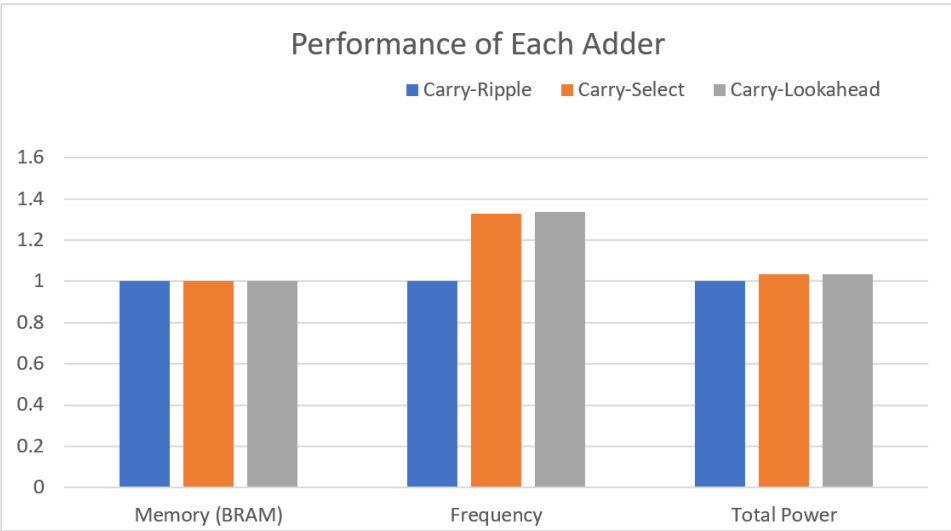

Figure 16: Plot of Performance of Each Adder

4 Answers to the 2 Postlab Questions

(a) **Compare the usage of LUT, Memory, and Flip-Flop of your bit-serial logic processor exercise in the IQT with your TTL design in Lab 3. Make an educated guess of the usage of these resources for TTL assuming the processor is extended to 8-bit. Which design is better, and why?**

	TTL 4-bit logic processor	System Verilog 4-bit logic processor	System Verilog 8-bit logic processor
LUT	25	46	72
Memory	0	0	0
Flip-Flop	11	27	43

Figure 17: PERFORMANCE OF LOGIC PROCESSOR

- **LUT** TTL design for a 4-bit logic processor uses the fewest LUTs compared to System Verilog design (i.e., 46 LUTs for a 4-bit logic processor and 73 LUTs for an 8-bit logic processor). If the TTL design is assumed to extend to 8-bit, it is estimated that the design uses approximately twice as many LUTs as the TTL design for 4-bit. So, approximately 50 LUTs are needed. This is still less than the System Verilog design for the 8-bit logic processor (i.e., 72). The reason is that TTL designs rely on discrete logic elements.
- **Memory** All the designs use 0 memory. They do not use dedicated memory resources such as BRAM. The memory is most likely integrated into the logic. The extended TTL for 8-bit also seems to have 0 memory usage since the design concept is essentially the same as 4-bit except for the bit width and the stage used.
- **Flip-Flop** The TTL design for a 4-bit logic processor uses fewer flip-flops than its System Verilog design. Extending the TTL design to 8-bit may cause flip-flops to double to about 22 flip-flops. This number is still less than the System Verilog design for 8-bit processors. The flip-flops of the TTL design are fewer than those of the System Verilog design because the TTL design is simpler and manual, while the System Verilog design is more complex and optimized.

In terms of resource utilization, the TTL design is better than the System-Verilog design. However, in its implementation, System Verilog is more flexible and faster (due to reduced propagation delay compared to discrete TTL components). So System Verilog design is a good choice for more complex and large designs.

(b) **For the adders, refer to the Design Resources and Statistics in IQT.16-18 and complete the following design statistics table for each adder. This is more comprehensive than the above design analysis and is required for every SystemVerilog circuit.**

	Carry-Ripple Adder (CRA)	Carry-Lookahead Adder (CLA)	Carry-Select Adder (CSA)
LUT	113	128	125
DSP	0	0	0
Memory (BRAM)	0	0	0
Flip-Flop	104	105	105
Frequency	59.45 MHz	79.42 MHz	78.9 MHz
Static Power	98.55 mW	98.57 mW	98.57 mW
Dynamic Power	4.01 mW	6.62 mW	6.89 mW
Total Power	156.86 mW	162.26 mW	162.62 mW

Figure 18: Design Statistics Table of Adders

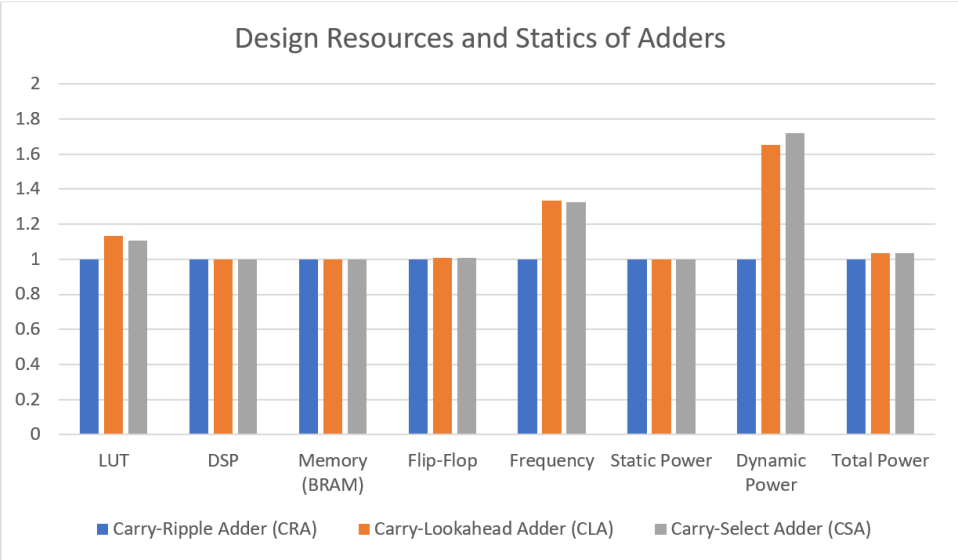

Figure 19: Design Statistics Plot of Adders

(c) **Observe the data plot and provide an explanation to the data, i.e., does each resource breakdown comparison from the plot make sense? Are they complying with the theoretical design expectations, e.g., the maximum operating frequency of the carry-lookahead adder is higher than the carry-ripple adder? Which design consumes more power than the other as you expected, why?**

The data meets expectations in terms of the number of LUTs, with CRA using less area compared to CLA and CSA. CLA has a higher area because it requires additional logic to compute the generated and propagated signals. In addition, the carry-lookahead unit also increases the area used. The CSA area is slightly smaller than CLA because of its optimized parallel structure.

All designs have 0 DSP and memory (BRAM) usage. This data is in line with expectations, as designs do not require memory (BRAM) to store data. Furthermore, DSP is not needed since the design is not a really complex mathematical operation but a combinational logic design.

The data aligns with the theoretical design expectations, where CRA is the slowest adder. Sequential carry-bit propagation increases CRA's propagation delay. CLA is the fastest adder, and CSA is in the middle. CLA has faster computation due to its "carry bits prediction" implementation. CSA is slightly slower than CLA due to the use of multiplexers and the lack of a "predict" of the carry bit, like CLA.

As expected, CLA consumes more power than CSA. The total power of CSA design is higher due to parallel computation and multiplexers.

5 Conclusion

5.1 Any Bugs and Countermeasures Taken During This Lab

- Due to an inadvertent oversight, we forgot to extend the bit-width of output Data_Out in the 8-bit logic processor assignment. We solved this problem by changing the bit-width to output logic [7 : 0] Data_Out.
- When testing Lab 4 (i.e., adders part) on the DE2-115 board, Carry-Lookahead Adder did not give the correct output. After reviewing the code in the .sv file, we managed

to identify the cause. It turned out that the implementation of propagation (P) and generation (G) logic did not match the block diagram provided in the lab manual, and there was an error in the Boolean expression that we typed. Finally, the output of CLA was correct after we redesigned it according to the block diagram and Boolean expression in the lab manual.

- Since we are not yet familiar with System Verilog, most of the difficulties we experienced were technical errors in implementing the language. We encountered several syntax errors while working with Lab 4 in the Quartus application. However, all bugs can be resolved by following the instructions in the error messages in Quartus and studying the System Verilog tutorial in the Lab Manual Complete V20.0_ZJUI Modified file.

5.2 Suggestions for Improvement and Feedback

The Lab Manual and other materials are very clear, straightforward, and informative. The guides and instructions are easy to understand and leave no ambiguity in their interpretation. Overall, this lab is very useful and has improved our knowledge of System Verilog and FGPA. The tutorials and guidelines provided were thoroughly detailed and helpful in completing this lab.